

Universidade do Minho
Escola de Engenharia
Mestrado em Engenharia Informática

Unidade Curricular de Administração de Bases de Dados

Ano Letivo de 2025/2026

Alexandre Miranda	**João Cunha**	**João Macedo**	**Mário Silva**
pg61507	pg60272	pg60274	pg60282

janeiro, 2026

ABD

Alexandre Miranda pg61507
João Cunha pg60272
João Macedo pg60274
Mário Silva pg60282

janeiro, 2026

Índice

1. Introdução 1

2. Carga Analítica 2

2.1. Interrogação Analítica 1 2

2.1.1. Desempenho *baseline* 2

2.1.2. Análise do Plano de Execução (*baseline*) 3

2.1.3. Indexação Parcial (`type,duration desc`) 4

2.1.4. Reestruturação da Lógica de Agregação 4

2.1.5. Resultados Finais 5

2.1.6. Comparação com DuckDB 6

2.2. Interrogação Analítica 2 7

2.2.1. Desempenho *baseline* 7

2.2.2. Análise do Plano de Execução (*baseline*) 8

2.2.3. Estratégias de Otimização Implementadas 9

2.2.4. Resultados Finais 11

2.2.5. Comparação com DuckDB 11

2.3. Interrogação Analítica 3 13

2.3.1. Desempenho baseline 13

2.3.2. Análise do plano de execução 13

2.3.3. Indexação de colunas 14

2.3.4. Reestruturação da lógica da pesquisa 15

2.3.5. Resultados finais 16

2.3.6. Comparação com DuckDB 16

2.4. Interrogação Analítica 4 18

2.4.1. Desempenho *baseline* 18

2.4.2. Análise do Plano de Execução (baseline) 18

2.4.3. Indexação da Coluna `username` 19

2.4.4. Reestruturação da Lógica de Exclusão (NOT EXISTS) 20

2.4.5. Resultados Finais 21

2.4.6. Comparação com DuckDB 21

3. Carga Transacional 23

3.1. Optimização através de Índices 23

3.2. Optimização de Parâmetros de Configuração 25

3.2.1. shared_buffers 25

3.2.2. work_mem 27

3.2.3. effective_cache_size 28

3.2.4. Paralelismo 30

3.2.5. fsync 31

3.2.6. synchronous_commit 32

4. Conclusão 34

Lista de Figuras

Figura 1 Plano de Execução Q1 (versão original). 3
Figura 2 Plano de Execução da Q1 (versão otimizada) 5
Figura 3 Plano de execução inicial da Q2 8
Figura 4 Plano de execução final da Q2 11
Figura 5 Plano de Execução Q3 (versão original). 14
Figura 6 Plano de Execução Q3 (versão otimizada). 16
Figura 7 Plano de execução *baseline* da *query* 4. 19
Figura 8 Plano de execução após a criação do índice `idx_star_username`. 20
Figura 9 Plano de execução final com NOT EXISTS. 21
Figura 10 Comparação do plano de execução da query `SearchRepositories` antes e depois da criação do índice GIN na coluna `description`. 23
Figura 11 Impacto do índice composto `idx_repository_owner_pushed_at` na query `GetUserInfo`, eliminando a operação de ordenação (Sort). 24
Figura 12 Optimização da junção na query `GetRepositoriesByLanguage` através da indexação da chave estrangeira `language_id`. 24
Figura 13 Evolução do *throughput* em função do `shared_buffers`. 26
Figura 14 Impacto do aumento de work_mem (4MB, 16MB e 32MB) no tempo de execução das queries. 28

Lista de Tabelas

Tabela 1 Comparação de desempenho entre a versão *baseline* e a versão otimizada. 6
Tabela 2 Comparação de desempenho Q1: PostgreSQL otimizado vs. DuckDB. 6
Tabela 3 Comparação de tempos de execução: *baseline* vs. versão otimizada. 11
Tabela 4 Comparação de desempenho: PostgreSQL otimizado vs. DuckDB. 12
Tabela 5 Comparação de desempenho: PostgreSQL optimizado vs. DuckDB sem optimização. 21
Tabela 6 Comparação de *performance*: baseline vs. versão com índices. 25
Tabela 7 Informação de memória disponível no sistema. 25
Tabela 8 Impacto da alocação de `shared_buffers` no desempenho transacional. 26
Tabela 9 Resultados do EXPLAIN ANALYZE com work_mem = 4MB. 27
Tabela 10 Tempos de execução da query `SearchRepositories` para diferentes tamanhos de cache. . . 29
Tabela 11 Impacto do `max_parallel_workers_per_gather` no tempo de execução da query GetRepositoriesByLanguage. 30
Tabela 12 Impacto de `max_parallel_workers_per_gather` e `max_parallel_workers` no desempenho transacional. 31
Tabela 13 Impacto da desativação do *fsync* no desempenho transacional. 31
Tabela 14 Tempo de resposta por query com fsync=off. 32
Tabela 15 Impacto do synchronous_commit no desempenho transacional. 33
Tabela 16 Tempo de resposta por query com synchronous_commit=off. 33

1. Introdução

No âmbito da Unidade Curricular de Administração de Bases de Dados, este relatório descreve o processo de otimização e avaliação de desempenho de um sistema de gestão de dados em PostgreSQL. O projeto simula um cenário real de utilização intensiva de uma plataforma de gestão de repositórios de código ("GitBench"), utilizando um *dataset* de metadados extraído do GitHub.

O objetivo principal deste trabalho prático é maximizar a eficiência do sistema perante dois tipos distintos de carga de trabalho: uma carga analítica (OLAP), caracterizada por consultas complexas de agregação e análise temporal, e uma carga transacional (OLTP), que simula operações concorrentes de *backend*, como criação de repositórios, gestão de *tickets* e interação de utilizadores.

O desenvolvimento do trabalho ocorreu numa infraestrutura computacional fixa na Google Cloud, numa instância N2 com 8 vCPUs, 16 GB de RAM e 500 GB de armazenamento SSD.

2. Carga Analítica

2.1. Interrogação Analítica 1

2.1.1. Desempenho *baseline*

A primeira interrogação analítica (Q1) procura identificar os 100 *tickets* com maior tempo de encerramento (*merged/closed*) para cada tipo de *ticket* (*issue* e *pull request*). Estes 100 *tickets* têm ainda de cumprir certas condições para serem considerados.

```
SELECT repository_id, ticket_id, created_by, type, title, time_to_close
FROM (
    SELECT t.repository_id, t.ticket_id, t.created_by, t.type, t.title,
t.time_to_close,
        rank() OVER(PARTITION BY type ORDER BY time_to_close desc) AS rank
    FROM (
        SELECT repository_id, ticket_id, created_by, type,
            closed_at - opened_at AS time_to_close, title
        FROM ticket
        WHERE (status = 'merged' OR status = 'closed')
            AND length(data) > 300
            AND (title || data) LIKE '%perf%'
    ) t
    JOIN repository r ON r.id = t.repository_id
    JOIN users u ON u.username = t.created_by
    WHERE r.owner <> u.username
        AND NOT r.is_archived
)
WHERE rank <= 100
ORDER BY 4, 6 DESC;
```

Listagem 1: Código SQL da *query* Q1 (versão original).

Esta *query* consiste em 4 fases principais:

- **Filtragem inicial**:

Nesta primeira fase, a subquery mais interna filtra os *tickets* cujo campo `data` apresenta um comprimento superior a 300 caracteres, que se encontrem no estado `closed` ou `merged`, e nos quais a palavra `perf` esteja presente na concatenação do `title` com `data`. Nesta etapa, calcula-se também o `time_to_close`, que será utilizado posteriormente.

- **Agregação de dados:**

Com a lista de *tickets* já filtrada, é realizado um `JOIN` com as chaves primárias das tabelas `repository` e `users`, com o objetivo de remover os tickets criados pelos *owners* do respetivo repositório, bem como os associados a repositórios arquivados.

- **Separação e *Ranking***:

Já com os dados limpos e filtrados com a lógica de negócio pretendida, utilizamos uma *window function* para dar um número a cada ticket que tenha passado a fase de filtragem, com base no `time_to_close`

calculado na primeira fase. Este processo é realizado de forma independente para cada valor de `type`, garantindo que cada tipo de *ticket* possui a sua própria numeração.

- **Filtragem final e Ordenação:**

Por fim, são selecionados os 100 *tickets* com pior desempenho, tanto do tipo `issue` como do tipo `pull_request` (visto que cada tipo possui uma numeração independente, o resultado total corresponde a 200 *tickets* com `WHERE rank <= 100`), e ordenamos em ordem ao `type` e ao `time_to_close`.

Com o objetivo de ter uma comparação base, foram medidos os tempos da *query* original, correndo 3x um *script* que repete a *query* 5x.

Nº Execução	tempo médio (s)
1	12.585 s
2	12.584 s
3	12.570 s

2.1.2. Análise do Plano de Execução (*baseline*)

Na Figura 1 apresenta-se o plano de execução da Q1, obtido através do comando `EXPLAIN ANALYZE` e analisado com o auxílio da ferramenta **explain.dalibo.com**.

	time	rows	estim	cost	loops	filter	heap	
#1	0.024	200	88 × ▲	176.17				Subquery Scan on unnamed_subquery
#2	49.930	200	88 × ▲	352.34				└ WindowAgg
#3	221.266	989,065	56 × ▼	3,033.46				└ Gather Merge
#4	566.948	~989,064	45 × ▼	489.63	3			└ Sort by ticket.type, ((ticket.closed_at - ticket.opened_at)) DESC
#5	12.488	~989,064	45 × ▼	31,265.80	3			└ Nested Loop
#6	45.748	~989,064	45 × ▼	51,569.47	3			├ Nested Loop
#7	6,331.327	~1,063,356	45 × ▼	950,234.08	3	89%		│ ├ Parallel Seq Scan on ticket
#8	2,835.619	~1,063,357		6.61	1,063,357			│ └ Index Scan on repository as r using repository_pkey
#9	2,967.195	~989,065		4.25	989,065		0	└ Index Only Scan on users as u using users_pkey

Figura 1: Plano de Execução Q1 (versão original).

- **Gargalo Principal - Estratégia de processamento (Agregação):**

Como podemos ver na Figura 1, a causa predominante da lentidão na execução da *query* em análise é o *Parallel Seq Scan* dentro de um *Nested Loop*, onde o volume de dados (~1M de linhas) causa uma demora de 6.3s no processamento dos dados provenientes da tabela *tickets*. Isto evidencia uma limitação na estrutura da *query*, que processa um volume muito elevado de dados apenas para descartar a maioria deles na fase de filtragem com o `WHERE rank <= 100` após a *window function* `rank() OVER(PARTITION BY type ORDER BY time_to_close desc)`.

Isto afeta também os *Index Scans*, que apesar de individualmente serem rápidos, acabam por atrasar a query ao demorar ~3s cada (*Index Scan repository* e *Index Only Scan users*) e provoca um *memory spill* (external sort), fazendo com que o PostgreSQL passe a realizar leituras e ordenações em disco a partir do momento em que o limite de memória disponível (`work_mem`) é atingido.

- **Outros problemas de *performance*:**

O cálculo constante da variável `time_to_close`, bem como o processamento e a separação dos dados por cada valor de `type`, também contribuem para os tempos observados na tabela de resultados. Este processamento pode ser evitado por completo com algumas mudanças no schema e a criação de alguns índices.

2.1.3. Indexação Parcial (`type,duration desc`)

Com o objetivo de mitigar o custo de execução *baseline*, foi abordada a componente mais dispendiosa da *query*. Para isso, foram realizadas algumas alterações na tabela `ticket`, que visam simplificar e reestruturar a filtragem nesta tabela.

Neste caso, foi criado um índice parcial composto que reflete a ordem de classificação exigida pela Q1. Este índice é designado como parcial, uma vez que apenas abrange os registos que satisfazem a condição `WHERE status IN ('merged','closed')`. Foi tomada a decisão de apenas indexar linhas que cumpram esses quesitos para diminuir o tamanho do índice, maximizando a performance de leitura e diminuindo a perda de performance de futuras *queries* transacionais.

```
CREATE INDEX idx_ticket_analytic_rank
ON ticket (type, duration DESC)
WHERE status IN ('merged', 'closed');
```

Este índice permite que os dados aos quais vamos aceder já estejam ordenados, diminuindo assim algum custo na zona mais crítica. No entanto, para tirar partido desta otimização foi necessário reestruturar a *query* em si.

2.1.4. Reestruturação da Lógica de Agregação

A *query* original sofria em grande parte por causa de filtragem tardia. Lia cerca de 1 milhão de linhas, ordenava-as (muitas vezes em disco graças aos *memory slips*), calculava o `RANK` para todas, e só no final descartava a maioria dos dados.

Para resolver isto, a lógica foi invertida utilizando `CROSS JOIN LATERAL`. Em vez de processar o conjunto de dados inteiro de uma só vez, a nova *query* itera sobre os tipos de tickets conhecidos e, para cada tipo, consulta o índice para obter apenas os 100 primeiros registos (pré-ordenados como mencionado anteriormente).

Para não atrasar a pesquisa com um `SELECT DISTINCT` à tabela `ticket` inteira apenas para ir buscar os diferentes tipos, foi realizada a seguinte alteração no *schema*:

```
CREATE TYPE ticket_type_enum AS ENUM ('issue', 'pull_request');

CREATE TABLE ticket (
    repository_id integer,
    ticket_id integer,
    created_by varchar,
    type ticket_type_enum,
    status varchar,
    opened_at timestamp,
    closed_at timestamp,
    title varchar,
    data varchar,
    PRIMARY KEY (repository_id, ticket_id)
);
```

Isto garante a utilização de valores fixos na filtragem pelo `type`, o que nos poupa percorrer milhares de linhas de uma tabela para ir buscar alguns tipos. Os tipos do `ENUM` foram recolhidos através da seguinte *query*:

```
SELECT DISTINCT type FROM ticket;
```

Após esta alteração, partimos então para a reestruturação. O resultado final foi:

```sql
SELECT
     top_tickets.repository_id,
     top_tickets.ticket_id,
     top_tickets.created_by,
     top_tickets.type,
     top_tickets.title,
     top_tickets.time_to_close
 FROM (
     VALUES
         ('issue'::ticket_type_enum),
         ('pull_request'::ticket_type_enum)
 ) AS types(type)
 CROSS JOIN LATERAL (
     SELECT
         t.repository_id, t.ticket_id, t.created_by, t.type, t.title,
         t.duration as time_to_close
     FROM ticket t
     JOIN repository r ON r.id = t.repository_id
     JOIN users u ON u.username = t.created_by
     WHERE
         t.type = types.type
         AND t.status IN ('merged', 'closed')
         AND length(t.data) > 300
         AND (t.title || t.data) LIKE '%perf%'
         AND r.owner <> u.username
         AND NOT r.is_archived
     ORDER BY t.duration DESC
     LIMIT 100
 ) top_tickets
 ORDER BY 4, 6 DESC;
```

Listagem 2: Código SQL da Q1 (versão otimizada).

Esta abordagem revela-se eficiente, funcionando conceptualmente como um "ciclo for" ao nível da base de dados:

1. Seleciona o tipo **`issue`**.

2. Percorre o índice **`idx_ticket_analytic_rank`** e para imediatamente após encontrar 100 registos válidos (isto funciona pois o índice pré-ordena os dados).

3. Repete o processo para **`pull_request`**.

2.1.5. Resultados Finais

A combinação do índice ordenado com a cláusula `LIMIT` dentro do `LATERAL JOIN` permitiu transformar uma leitura sequencial massiva (*Seq Scan*) numa leitura indexada de precisão (*Index Scan*).

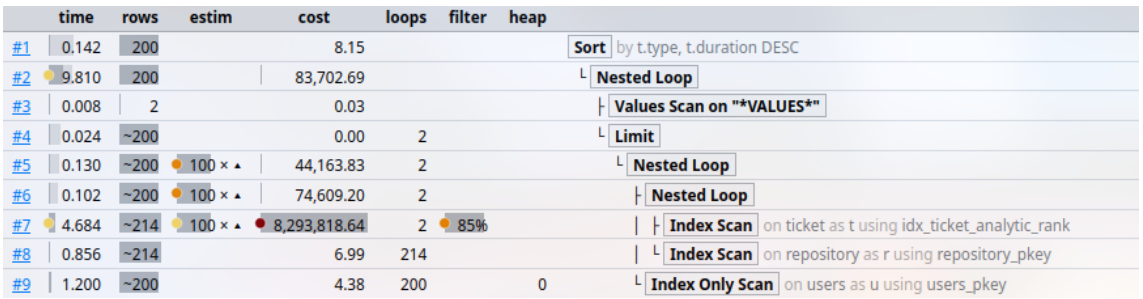


Figura 2: Plano de Execução da Q1 (versão otimizada)

Como se observa em **Figura 2**, o tempo de execução reduziu-se de aproximadamente 12.5s para valores na ordem dos milissegundos, uma vez que a base de dados agora lê apenas cerca de 200 a 300 linhas do disco, em vez de ~1 milhão.

De seguida, apresentam-se os resultados do tempo médio (s) para a Q1 otimizada, com o mesmo *script* utilizado para obter os resultados *baseline*. A melhoria de performance é superior a 800×.

N.º execução	tempo base (s)	tempo final (s)	melhoria
1	12.585 s	0.015 s	839x
2	12.584 s	0.015 s	839x
3	12.570 s	0.015 s	838x

Tabela 1: Comparação de desempenho entre a versão *baseline* e a versão otimizada.

2.1.6. Comparação com DuckDB

Por fim, comparou-se a solução otimizada no PostgreSQL com a execução da query original no DuckDB, utilizando o mesmo dataset.

N.ºexecução	PostgreSQL (otimizado)	DuckDB (sem otimização)	diferença
1	0.015 s	2.777 s	PG −99.5%
2	0.015 s	2.727 s	PG −99.5%
3	0.015 s	2.754 s	PG −99.5%

Tabela 2: Comparação de desempenho Q1: PostgreSQL otimizado vs. DuckDB.

A análise dos resultados revela uma disparidade significativa, com o PostgreSQL a ser aproximadamente 180 vezes mais rápido que o DuckDB. Esta diferença ilustra uma distinção fundamental entre as arquiteturas dos dois sistemas para este tipo de consulta específica:

1. PostgreSQL:

Graças à estratégia de *Index Scan* combinada com o `LATERAL`, o PostgreSQL acede diretamente às 100 entradas necessárias no índice e ignora os restantes milhões de registos.

2. DuckDB (Column-Store):

O DuckDB é otimizado para agregações massivas, lendo colunas inteiras de forma vetorial. No entanto, para resolver uma *query* de `RANK()` e `LIMIT`, o DuckDB é forçado a ler e processar a totalidade da coluna `type` e de calcular sempre o `time_to_close` para determinar a ordem correta antes de poder descartar os dados. Sem um índice ordenado (como a B-Tree do Postgres), o DuckDB paga o custo de processar o *dataset* inteiro para extrair apenas uma pequena fração dos dados.

Em conclusão, embora o DuckDB seja geralmente mais eficiente em leituras completas de tabelas (*Full Scans*), o PostgreSQL demonstra uma vantagem clara em cenários de elevada seletividade quando devidamente indexado e otimizado.

2.2. Interrogação Analítica 2

2.2.1. Desempenho *baseline*

A segunda interrogação analítica (Q2) identifica os 25 repositórios mais populares de um determinado mês, com base no número de estrelas recebidas. Adicionalmente, a consulta lista as três linguagens de programação predominantes em cada repositório.

```
SELECT id, full_name, languages, count
FROM (
    SELECT *, rank() OVER(PARTITION BY month ORDER BY count DESC) AS rank
    FROM (
        SELECT r.id AS id,
            r.owner || '/' || r.name AS full_name,
            date_trunc('month', s.starred_at) AS month,
            l.languages,
            count(s.repository_id) AS count
        FROM repository r
        LEFT JOIN star s ON s.repository_id = r.id
        LEFT JOIN (
            SELECT rl.repository_id, (array_agg(l.name ORDER BY rl.size DESC))[:3]
AS languages
            FROM repository_languages rl
            JOIN language l ON l.id = rl.language_id
            GROUP BY 1
        ) l ON l.repository_id = r.id
        GROUP BY 1, 2, 3, 4
    )
)
WHERE rank <= 25
    AND month = '2024-12-01'
ORDER BY count DESC;
```

Listagem 3: Código SQL da *query* Q2 (versão original).

A lógica da consulta consiste em três etapas principais:

1. **Agregação de estrelas e linguagens:**

Para cada repositório (`repository`), a *query* contabiliza o número de estrelas (`count(s.repository_id)`) por mês e agrega as linguagens de programação mais relevantes (`array_agg(l.name ORDER BY rl.size DESC)[:3]`)

2. **Classificação por popularidade mensal:**

Utilizando a função `rank() OVER(PARTITION BY month ORDER BY count DESC)`, os repositórios são classificados em cada mês de acordo com o número de estrelas.

3. **Filtragem dos top 25 por mês:**

Por fim, a consulta devolve apenas os repositórios com `rank <= 25` e do mês de referência (`month = "2024-12-01"`), ordenados pelo número de estrelas em ordem decrescente.

A versão original apresentou tempos de execução elevados devido à necessidade de processar dados de todos os meses antes de filtrar.

A tabela seguinte resume os tempos médios obtidos:

argumento (mês)	tempo médio (s)
2024-12-01	16.921 s
2024-11-01	15.255 s
2024-10-01	15.144 s

2.2.2. Análise do Plano de Execução (*baseline*)

O plano de execução revela que a estratégia adotada pelo otimizador sofre de ineficiências estruturais significativas, como está retratado na seguinte imagem extraída do **explain.dalibo.com**, resultando num tempo total de execução de aproximadamente 15.7 segundos.

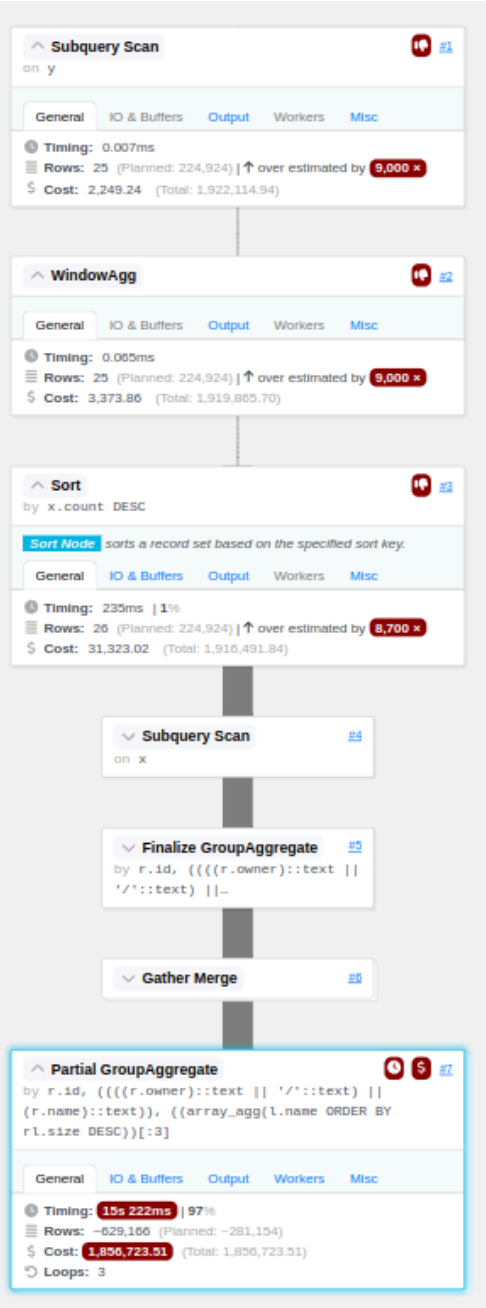


Figura 3: Plano de execução inicial da Q2

A análise dos nós permite identificar hierarquicamente os problemas:

1. **Gargalo principal – agregação global e leitura excessiva:**

A causa predominante da lentidão reside na estratégia de "Filtragem Tardia" (*Late Filtering*). O nó `Finalize GroupAggregate` (e os seus processos filhos `Partial GroupAggregate`) processa a contagem de estrelas para **todos** os repositórios da base de dados, abrangendo todo o histórico temporal. O plano mostra que o filtro `WHERE month = '2024-12-01'` e o limite `rank <= 25` são aplicados apenas **após** o processamento pesado de milhões de linhas. Este comportamento obriga o CPU a realizar cálculos de agregação complexos em dados que serão descartados imediatamente a seguir.

2. **Gargalo secundário – extravasamento de memória (*disk spill*):**

Devido ao enorme volume de dados gerado pela agregação global, a memória de trabalho alocada (`work_mem`) foi insuficiente para realizar a ordenação em RAM. O nó `Sort`, necessário para a função `rank()`, forçou a escrita de 61 MB em disco:

***Sort Method*:** `external merge Disk: 61152kB`.

Escrever e ler 61 MB de dados temporários no disco introduz uma latência significativa de I/O, bloqueando o pipeline de execução enquanto os dados são serializados e deserializados.

3. **Impacto no I/O (*buffers*):**

A ineficiência da estratégia reflete-se no consumo massivo de largura de banda de armazenamento:

- **leituras físicas (*read*):** foram lidas 498.887 páginas do disco. Considerando páginas de 8kB, isto equivale a transferir **4 GB de dados** do disco para a memória para resolver uma única consulta.
- **leituras lógicas (*shared hit*):** o valor de 27 milhões de *hits* na cache indica que o CPU teve de revisitar as mesmas páginas de memória repetidamente para processar as junções e agregações não otimizadas.

Em conclusão, a query sofre de um problema de escala: o custo de execução cresce linearmente com o tamanho total da base de dados, independentemente de estarmos a pedir dados de apenas um mês.

2.2.3. Estratégias de Otimização Implementadas

A *query* original apresentava tempos de execução elevados devido ao cálculo de agregações e junções (JOINs) em todo o conjunto de dados antes da filtragem. A versão otimizada reduziu o tempo de execução drasticamente (de ~15s para ~1s) através de três alterações estruturais:

1. **Filtragem SARGable (Search ARGumentable):**

A cláusula original `date_trunc('month', s.starred_at)` impedia o uso de índices, forçando uma leitura sequencial (Seq Scan). Substituiu-se por um intervalo de datas explícito (`starred_at >= X AND starred_at < Y`), permitindo que o PostgreSQL utilizasse o índice para saltar diretamente para os registos relevantes.

2. **Index Only Scan (índice de cobertura):**

Criou-se o índice `idx_star_optimize` com a data como primeira coluna para suportar o filtro e o ID como segunda para a contagem:

```
CREATE INDEX CONCURRENTLY IF NOT EXISTS idx_star_optimize
ON star (starred_at, repository_id);
```

Isto permite que o banco responda à agregação lendo apenas a estrutura do índice em memória (*Index Only Scan*), eliminando a necessidade de aceder à tabela principal (*Heap Fetch*).

- **3. Adiamento de joins e indexação ordenada:**

Foi utilizada uma CTE para isolar os ID's dos repositórios no top 25 antes de qualquer join. Para a subconsulta de linguagens, criou-se um índice que já incorpora a ordenação necessária.

```
CREATE INDEX CONCURRENTLY IF NOT EXISTS idx_repo_lang_optimize
ON repository_languages (repository_id, size DESC);
```

Isto permitiu que a pesquisa do top 3 de linguagens fosse instantânea, uma vez que o índice já fornece os dados ordenados por size DESC, evitando operações de ordenação dispendiosas.

A versão final da *query* ficou estruturada da seguinte forma:

```sql
WITH top_stars_filtered AS (
    SELECT
        repository_id,
        count(*) AS count
    FROM star s
    WHERE s.starred_at >= '2024-12-01'::date
      AND s.starred_at < ('2024-12-01'::date + INTERVAL '1 month')
    GROUP BY repository_id
    ORDER BY count DESC
    LIMIT 25
)
SELECT
    t.repository_id AS id,
    r.owner || '/' || r.name AS full_name,
    '2024-12-01 00:00:00+00'::timestamptz AS month,
    (
        SELECT (array_agg(l.name ORDER BY rl.size DESC))[:3]
        FROM repository_languages rl
        JOIN language l ON l.id = rl.language_id
        WHERE rl.repository_id = t.repository_id
    ) AS languages,
    t.count
FROM top_stars_filtered t
JOIN repository r ON r.id = t.repository_id
ORDER BY t.count DESC;
```

Listagem 4: Código SQL da *query* Q2 (versão otimizada).

2.2.4. Resultados Finais

O impacto das otimizações foi significativo. A utilização do *Index Only Scan* eliminou a leitura da tabela star do disco e o *Deferred Join* evitou o processamento desnecessário de linguagens.

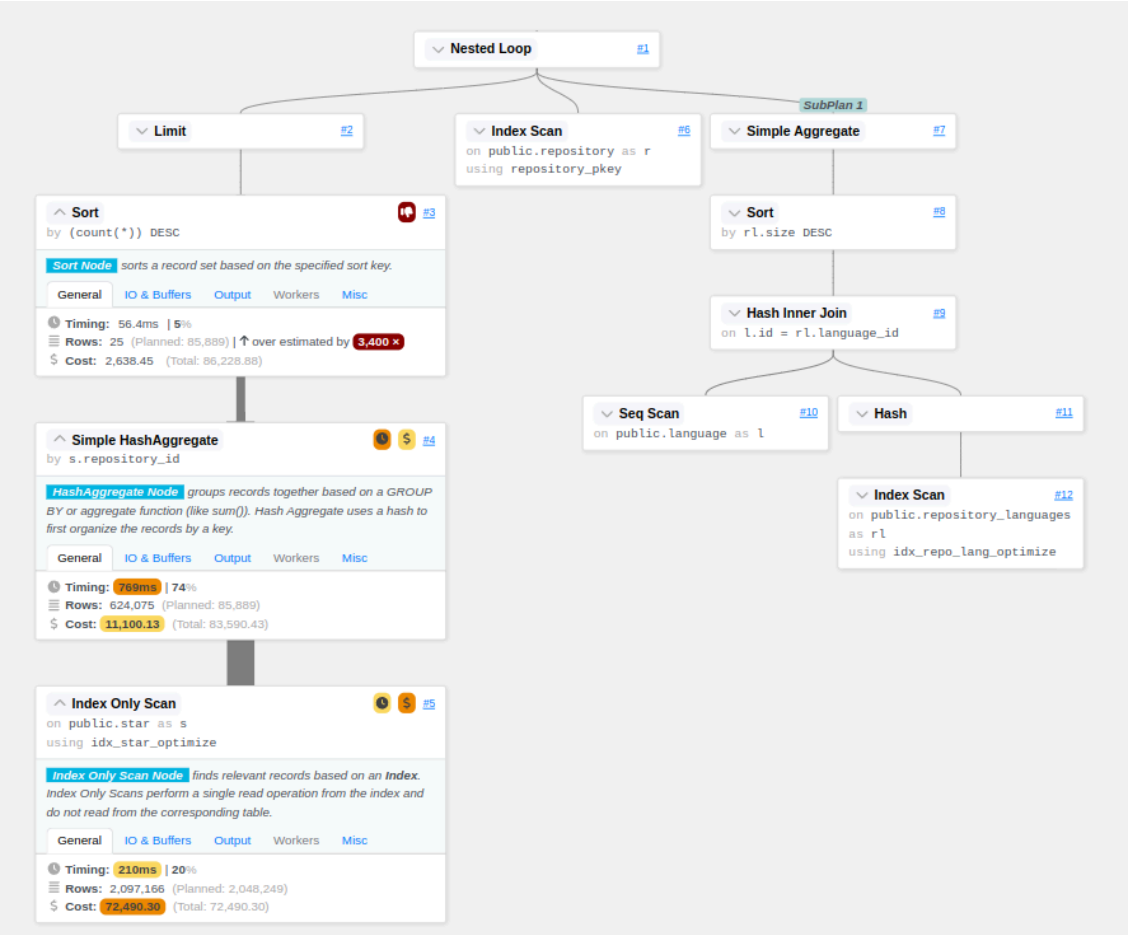


Figura 4: Plano de execução final da Q2

A tabela seguinte compara os tempos médios entre a versão original e a versão otimizada:

agrumento (mês)	**tempo base** (s)	**tempo otimizado** (s)
2024-12-01	16.921	**1.025**
2024-11-01	15.255	**0.861**
2024-10-01	15.144	**0.795**

Tabela 3: Comparação de tempos de execução: *baseline* vs. versão otimizada.

A redução de ~15.7s para ~0.9s representa uma melhoria de performance superior a 17×.

2.2.5. Comparação com DuckDB

Os testes no DuckDB foram executados sobre o mesmo conjunto de dados para comparar a abordagem otimizada do PostgreSQL contra a performance da query original no DuckDB. A Tabela 4 apresenta os resultados obtidos.

argumento	**PostgreSQL** (otimizado)	**DuckDB** (sem otimização)	**diferença**
2024-12-01	1.025 s	7.605 s	PG −86.5%
2024-11-01	0.861 s	7.637 s	PG −88.7%
2024-10-01	0.795 s	7.604 s	PG −89.5%

Tabela 4: Comparação de desempenho: PostgreSQL otimizado vs. DuckDB.

A análise dos resultados revela que o PostgreSQL otimizado é entre **7 a 9 vezes mais rápido** que o DuckDB neste cenário. A diferença de comportamento explica-se pelas estruturas de acesso aos dados:

1. **PostgreSQL (*Index Scan* + Agregação):**

O tempo de execução do PostgreSQL (0.8s a 1.0s) é dominado pelo custo de CPU da operação `HashAggregate`. Embora o índice `idx_star_optimize` permita isolar rapidamente o mês correto (evitando ler a tabela toda), o banco ainda precisa de contar e agrupar individualmente cerca de 2 milhões de registos, o que consome o tempo observado.

2. **DuckDB (*Columnar Scan*):**

O DuckDB, sem um índice na coluna de datas, é forçado a varrer a coluna inteira para filtrar os registos do mês desejado. Embora a sua arquitetura colunar seja eficiente, o custo de ler e filtrar dezenas de milhões de datas supera o custo da agregação focada que o PostgreSQL consegue fazer graças ao índice.

Em suma, embora o DuckDB apresente um desempenho nativo robusto para cargas analíticas, os resultados demonstram que a indexação direcionada do PostgreSQL permite obter vantagens claras em cenários de elevada seletividade temporal.

2.3. Interrogação Analítica 3

A terceira interrogação analítica (Q3) visa agregar diferentes tipos de eventos (*commits*, *tickets*, criação de utilizadores, etc.) ocorridos num determinado intervalo temporal.. Estes eventos são posteriormente agrupados em intervalos regulares de tempo e contabilizados.

2.3.1. Desempenho baseline

A consulta original é apresentada na seguinte imagem:

```
SELECT date_bin('15 minutes', date, '2009-01-01') AS interval, event, count(*) AS
count
FROM (
    SELECT 'commit' AS event, created_at AS date
    FROM commit
    UNION ALL
    SELECT 'ticket_opened', opened_at
    FROM ticket
    UNION ALL
    SELECT 'ticket_closed', closed_at
    FROM ticket
    WHERE closed_at IS NOT NULL
    UNION ALL
    SELECT 'repository', created_at
    FROM repository
    UNION ALL
    SELECT 'user', created_at
    FROM users
    UNION ALL
    SELECT 'star', starred_at
    FROM star
)
WHERE date BETWEEN '2022-01-01' AND '2025-01-01'
GROUP BY 1, 2
ORDER BY 1, 2;
```

Listagem 5: Código SQL da *query* Q3(versão original).

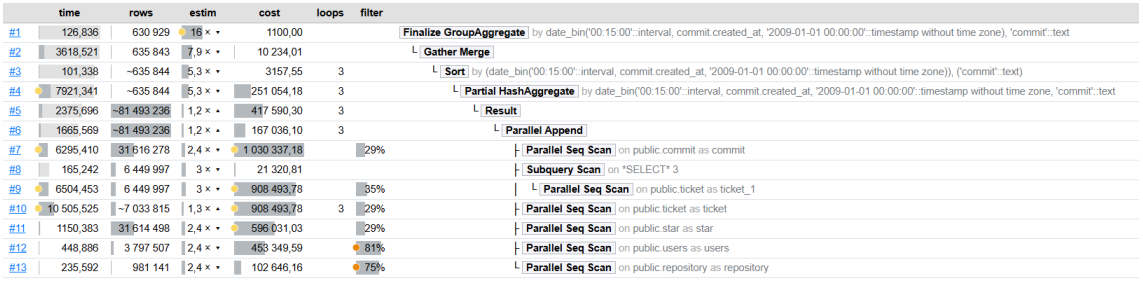
Esta versão reúne vários eventos fazendo uso do `UNION ALL`. Depois disso, filtra-os e agrupa-os em blocos de 15 minutos.

As medições iniciais evidenciaram constrangimentos de desempenho significativos, mais notoriamente no longo tempo de execução. Na seguinte tabela estão representadas as médias dos tempos de execução (média obtida através de10 execuções) para diversos intervalos de tempo e com diferentes bins.

intervalo de tempo	bin	tempo médio(s)
2022-01-01 a 2025-01-01	15 minutos	35.122 s
2022-01-01 a 2025-01-01	60 minutos	34.023 s
2020-01-01 a 2020-06-01	15 minutos	32.222 s

2.3.2. Análise do plano de execução

Para encontrarmos os principais entraves a um melhor funcionamento das pesquisas, gerou-se e analisou-se o plano de execução através do *EXPLAIN ANALYZE*.



	time	rows	estim	cost	loops	filter	
#1	126,836	630 929	16 ×	1100,00			Finalize GroupAggregate by date_bin('00:15:00'::interval, commit.created_at, '2009-01-01 00:00:00'::timestamp without time zone), 'commit'::text
#2	3618,521	635 843	7,9 ×	10 234,01			└ Gather Merge
#3	101,338	~635 844	5,3 ×	3157,55	3		└ Sort by (date_bin('00:15:00'::interval, commit.created_at, '2009-01-01 00:00:00'::timestamp without time zone)), ('commit'::text)
#4	7921,341	~635 844	5,3 ×	251 054,18	3		└ Partial HashAggregate by date_bin('00:15:00'::interval, commit.created_at, '2009-01-01 00:00:00'::timestamp without time zone, 'commit'::text
#5	2375,696	~81 493 236	1,2 ×	417 590,30	3		└ Result
#6	1665,569	~81 493 236	1,2 ×	167 036,10	3		└ Parallel Append
#7	6295,410	31 616 278	2,4 ×	1 030 337,18		29%	├ Parallel Seq Scan on public.commit as commit
#8	165,242	6 449 997	3 ×	21 320,81			├ Subquery Scan on *SELECT* 3
#9	6504,453	6 449 997	3 ×	908 493,78		35%	│ └ Parallel Seq Scan on public.ticket as ticket_1
#10	10 505,525	~7 033 815	1,3 ×	908 493,78	3	29%	├ Parallel Seq Scan on public.ticket as ticket
#11	1150,383	31 614 498	2,4 ×	596 031,03		29%	├ Parallel Seq Scan on public.star as star
#12	448,886	3 797 507	2,4 ×	453 349,59		81%	├ Parallel Seq Scan on public.users as users
#13	235,592	981 141	2,4 ×	102 646,16		75%	└ Parallel Seq Scan on public.repository as repository

Figura 5: Plano de Execução Q3 (versão original).

- **Gargalo Principal - Estratégia de processamento (Agregação):**

O plano mostra que o desempenho é dominado por múltiplas operações de *Parallel Seq Scan* sobre todas as tabelas envolvidas no `UNION ALL`, especialmente as tabelas maiores (`star`, `commit` e `users`). Cada uma delas percorre milhões de linhas apenas para aplicar filtros de datas antes de alimentar o *Parallel Append*.

Pontos principais observados:

1. **Gargalo principal**: um *Parallel Seq Scan* na tabela star que produz 15 807 249 linhas por nó com um tempo de 3815 ms a 10211 ms por processo.

```
Parallel Seq Scan on star … rows=5350832 … actual rows=15807249
```

2. **Gargalos Secundários**: as restantes tabelas são igualmente percorridas de forma integral e, devido ao seu elevado volume de dados, contribuem significativamente para o aumento do tempo total de execução.

```
Parallel Seq Scan on commit ...
actual rows=4516611 Rows Removed by Filter: 1911960
```

```
Parallel Seq Scan on users ...
actual rows=1898754 Rows Removed by Filter: 8101250
```

```
Parallel Seq Scan on ticket ...
actual rows=3224998 Rows Removed by Filter: 1775002
```

2.3.3. Indexação de colunas

Foram criados índices sobre as colunas temporais (`created_at`, `opened_at`, `closed_at`, `starred_at`) de todas as tabelas envolvidas na query uma vez que o plano de execução mostrava vários *Parallel Seq Scan* a milhões de linhas apenas para aplicar filtros de data.

Os índices criados evitam leituras completas das tabelas, diminuindo assim o custo das operações.

```
CREATE INDEX idx_commit_created_at
    ON commit (created_at);

CREATE INDEX idx_repository_created_at
    ON repository (created_at);

CREATE INDEX idx_star_starred_at
    ON star (starred_at);

CREATE INDEX idx_ticket_closed_at
    ON ticket (closed_at);

CREATE INDEX idx_ticket_opened_at
    ON ticket (opened_at);

CREATE INDEX idx_users_created_at
    ON users (created_at);
```

Listagem 10: Índices criados

2.3.4. Reestruturação da lógica da pesquisa

Na versão original, o filtro `WHERE date BETWEEN ...` era aplicado apenas após a união de todas as tabelas, forçando o processamento de um volume excessivo de dados que seriam descartados posteriormente.

A otimização consistiu em aplicar os filtros temporais e a agregação diretamente em cada *subquery*, **antes** do `UNION ALL`. Esta técnica reduz drasticamente a cardinalidade dos dados intermédios, diminuindo consequentemente o consumo de memória e a latência de I/O.

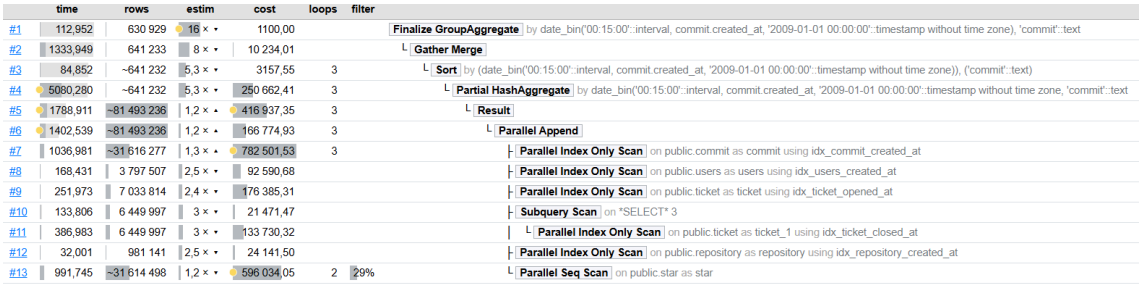
```
SELECT interval, event, SUM(count) AS count
FROM (
    SELECT
        date_bin('15 minutes', created_at, '2009-01-01') AS interval,
        'commit' AS event,
        COUNT(*) AS count
    FROM commit
    WHERE created_at BETWEEN '2020-01-01' AND '2020-06-01'
    GROUP BY 1

    UNION ALL

    SELECT
        date_bin('15 minutes', opened_at, '2009-01-01') AS interval,
        'ticket_opened' AS event,
        COUNT(*) AS count
    FROM ticket
    WHERE opened_at BETWEEN '2020-01-01' AND '2020-06-01'
    GROUP BY 1
    ...
    e no final
    ...
    GROUP BY interval, event
    ORDER BY interval, event;
```

Listagem 11: Excerto da query otimizada

2.3.5. Resultados finais



	time	rows	estim	cost	loops	filter	
#1	112,952	630 929	16 ×	1100,00			Finalize GroupAggregate by date_bin('00:15:00'::interval, commit.created_at, '2009-01-01 00:00:00'::timestamp without time zone), 'commit'::text
#2	1333,949	641 233	8 ×	10 234,01			└ Gather Merge
#3	84,852	~641 232	5,3 ×	3157,55	3		└ Sort by (date_bin('00:15:00'::interval, commit.created_at, '2009-01-01 00:00:00'::timestamp without time zone)), ('commit'::text)
#4	5080,280	~641 232	5,3 ×	250 662,41	3		└ Partial HashAggregate by date_bin('00:15:00'::interval, commit.created_at, '2009-01-01 00:00:00'::timestamp without time zone, 'commit'::text
#5	1788,911	~81 493 236	1,2 ×	416 937,35	3		└ Result
#6	1402,539	~81 493 236	1,2 ×	166 774,93	3		└ Parallel Append
#7	1036,981	~31 616 277	1,3 ×	782 501,53	3		├ Parallel Index Only Scan on public.commit as commit using idx_commit_created_at
#8	168,431	3 797 507	2,5 ×	92 590,68			├ Parallel Index Only Scan on public.users as users using idx_users_created_at
#9	251,973	7 033 814	2,4 ×	176 385,31			├ Parallel Index Only Scan on public.ticket as ticket using idx_ticket_opened_at
#10	133,806	6 449 997	3 ×	21 471,47			├ Subquery Scan on *SELECT* 3
#11	386,983	6 449 997	3 ×	133 730,32			│ └ Parallel Index Only Scan on public.ticket as ticket_1 using idx_ticket_closed_at
#12	32,001	981 141	2,5 ×	24 141,50			├ Parallel Index Only Scan on public.repository as repository using idx_repository_created_at
#13	991,745	~31 614 498	1,2 ×	596 034,05	2	29%	└ Parallel Seq Scan on public.star as star

Figura 6: Plano de Execução Q3 (versão otimizada).

Na tabela abaixo estão definidos os tempos médios de execução (média obtida através de 10 execuções) da pesquisa, com as atualizações e alterações realizadas. É importante referir também que, todos estes tempos, foram obtidos após ter sido realizada uma alteração tanto a nível de memória como na paralelização, através dos seguintes comandos:

```
SET work_mem = '10MB';
SET max_parallel_workers_per_gather = 6;
```

intervalo de tempo	bin	tempo médio(s)
2022-01-01 a 2025-01-01	15 minutos	6.011 s
2022-01-01 a 2025-01-01	60 minutos	5.552 s
2020-01-01 a 2020-06-01	15 minutos	0.351 s

Podemos concluir assim que, a reformulação da query 3, juntamente com a criação de índices, resultaram numa maior eficiência.

2.3.6. Comparação com DuckDB

Os testes no DuckDB foram efetuados sobre a query original e estão representados na tabela abaixo.

intervalo de tempo	bin	tempo médio(s)
2022-01-01 a 2025-01-01	15 minutos	1.428 s
2022-01-01 a 2025-01-01	60 minutos	1.066 s
2020-01-01 a 2020-06-01	15 minutos	0.109 s

A análise dos tempos de execução, após a otimização da query, mostra uma redução significativa do custo no PostgreSQL. No entanto, mesmo com estes ganhos, os resultados obtidos com o DuckDB revelam-se consistentemente superiores, apresentando tempos ainda mais baixos, como 1.428 s e 1.066 s para os intervalos maiores, e apenas 0.109 s para o intervalo mais curto.

Apesar da melhoria substancial no PostgreSQL (redução de 35s para 6s), o DuckDB apresenta tempos consistentemente inferiores (cerca de 4x mais rápido no intervalo maior).

Isto não reflete uma falha na otimização do PostgreSQL, mas sim uma diferença arquitetural fundamental:

1. **DuckDB (OLAP/colunar):** o armazenamento orientado a colunas permite ler apenas os dados necessários (neste caso, as datas) e vetorizar as operações de agregação, o que é ideal para este tipo de *query* analítica.

2. **PostgreSQL (OLTP/linha):** mesmo com índices, o motor orientado a linhas apresetna um *overhead* natural ao reconstruir tuplas completas e gerir a concorrência (MVCC), o que limita a velocidade máxima em agregações massivas.

Em suma, embora a otimização aplicada tenha melhorado substancialmente o desempenho no PostgreSQL, a arquitetura interna do DuckDB continua mais adequada a este tipo de tarefa analítica, justificando os seus tempos de execução inferiores.

2.4. Interrogação Analítica 4

A quarta interrogação analítica (Q4) visa gerar recomendações de repositórios para um dado utilizador. A lógica da consulta consiste em identificar "colegas de estrela" (utilizadores que deram estrela a repositórios em comum com o utilizador-alvo) e, subsequentemente, sugerir outros repositórios a que esses "colegas" também deram estrela.

A consulta exclui repositórios que o utilizador-alvo já marcou com estrela. O argumento da consulta é o `username`.

2.4.1. Desempenho *baseline*

A consulta original (*baseline*) utilizada para as medições é apresentada na figura abaixo. Esta versão utiliza a cláusula `NOT IN` para a lógica de exclusão.

```sql
SELECT r.id, r.owner || '/' || r.name, r.description
FROM users u
JOIN star s1 ON u.username = s1.username
JOIN star s2 ON s1.repository_id = s2.repository_id
    AND s1.username <> s2.username
JOIN star s3 ON s3.username = s2.username
JOIN repository r ON r.id = s3.repository_id
WHERE u.username = 'christopherharmon25'
    AND s3.repository_id NOT IN (
        SELECT repository_id
        FROM star
        WHERE username = 'christopherharmon25'
    )
GROUP BY 1, 2, 3
ORDER BY count(*) DESC
LIMIT 25;
```

Listagem 13: Código SQL da *query* Q4 (versão original).

A versão original da *query* apresentou tempos de execução muito elevados. A tabela resume os tempos médios obtidos através do script de medição automática (`arg_script.py`), executado cinco vezes por argumento.

argumento (*username*)	**tempo médio (s)**
`christopherharmon25`	19.119 s
`rdavila66`	10.425 s
`jenkinsbrandon52`	9.097 s

2.4.2. Análise do Plano de Execução (baseline)

Para diagnosticar a causa do baixo desempenho, gerou-se e analisou-se o plano de execução através do comando `EXPLAIN ANALYZE`. O plano (Figura 7) revelou que o desempenho era severamente degradado por múltiplas operações de `Parallel Seq Scan` (varredura sequencial) na tabela `star`, a maior tabela da base de dados.

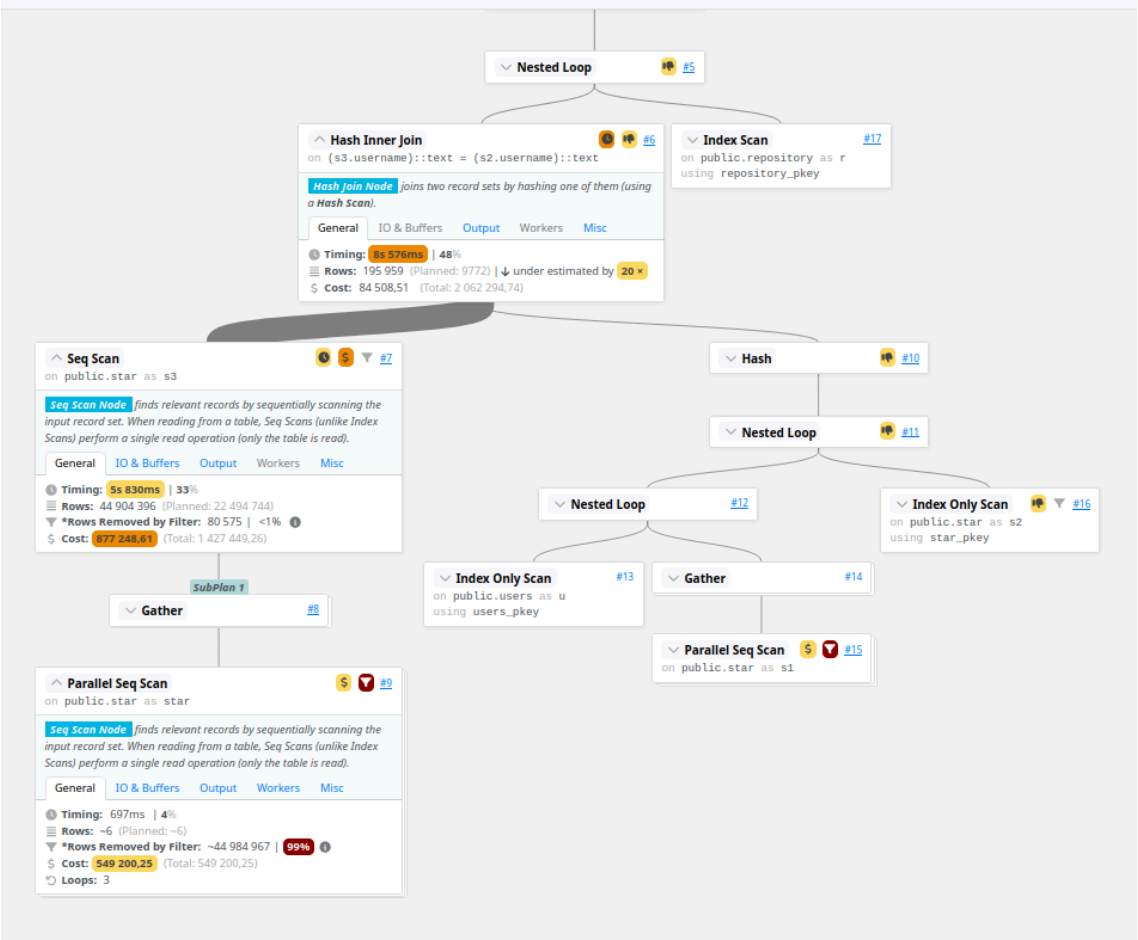


Figura 7: Plano de execução *baseline* da *query* 4.

Identificaram-se os seguintes pontos de contenção:

1. **Gargalo Principal:** um `Seq Scan` na tabela `star` (aliás *s3*), que demorava **6.5 segundos** a ler 45 milhões de linhas, apenas para alimentar um `Hash Join` cujo critério incluía a coluna `username`.
2. **Gargalos Secundários:** múltiplos `Parallel Seq Scan` na tabela `star` (aliás *s1* e no `SubPlan 1` do `NOT IN`) para filtrar pelo `username` do utilizador-alvo.

A origem principal desta ineficiência encontra-se na ausência de um índice adequado. O *schema* define a chave primária da tabela `star` como (repository_id,username), solução eficiente para pesquisas por `repository_id`, mas totalmente ineficaz para filtros ou *joins* cujo critério incide sobretudo sobre a coluna `username` (como ocorre em *s1* e *s3*)

2.4.3. Indexação da Coluna `username`

Para lidar com os múltiplos `Seq Scans`, a primeira estratégia consistiu na criação de um índice B-tree na coluna `username` da tabela `star`.

```
CREATE INDEX idx_star_username ON star (username);
```

A introdução deste índice originou uma melhoria de desempenho significativa, reduzindo o tempo de execução para o utilizador `christopherharmon25` de 19.1s para 2.8s (aproximadamente 7× mais rápido). O novo plano de execução (Figura 8) confirmou que todos os `Seq Scan` ligados à coluna `username` passaram a ser substituídos por `Index Scan` de elevada rapidez.

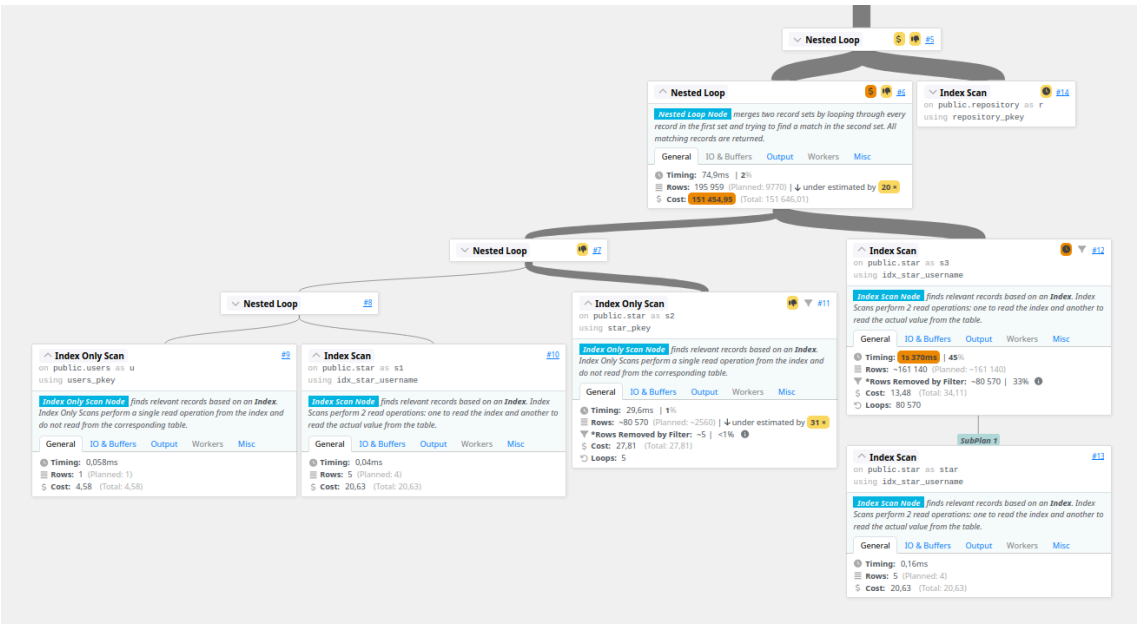


Figura 8: Plano de execução após a criação do índice `idx_star_username`.

Apesar da melhoria, a análise deste novo plano revelou o gargalo seguinte: um `Nested Loop` executado 80 570 vezes. Além disso, o plano continuava a recorrer a um `Sort` para disco (*external merge*), sinalizando ineficiências na lógica de exclusão com `NOT IN`.

2.4.4. Reestruturação da Lógica de Exclusão (NOT EXISTS)

O segundo problema identificado resultava da utilização de `NOT IN`, uma construção que o *planner* do *PostgreSQL* raramente trata de forma eficiente, resultando num Hashed SubPlan.

A lógica da consulta passou a recorrer a `NOT EXISTS`, uma construção que o *PostgreSQL* gere com maior eficiência. Esta alteração, combinada com o índice já existente, reduziu o tempo de execução de 2.8s para 1.92s.

```
SELECT r.id, r.owner || '/' || r.name, r.description
FROM users u
JOIN star s1 ON u.username = s1.username
JOIN star s2 ON s1.repository_id = s2.repository_id
    AND s1.username <> s2.username
JOIN star s3 ON s3.username = s2.username
JOIN repository r ON r.id = s3.repository_id
WHERE u.username = 'christopherharmon25'
AND NOT EXISTS (
    SELECT 1
    FROM star s_exclude
    WHERE s_exclude.repository_id = s3.repository_id
    AND s_exclude.username = 'christopherharmon25'
)
GROUP BY 1, 2, 3
ORDER BY count(*) DESC
LIMIT 25;
```

O plano de execução final (Figura 9) demonstra o sucesso desta abordagem. O *SubPlan* de `NOT IN` encontra-se agora substituído por um `Merge Join (Anti)` altamente eficiente, que realiza a exclusão em apenas 42 ms.

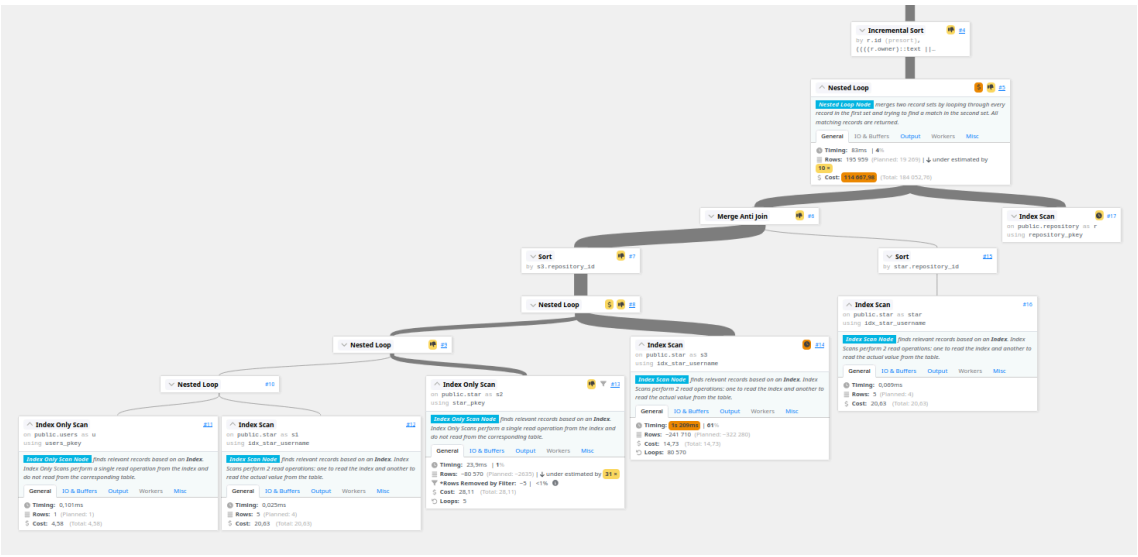


Figura 9: Plano de execução final com NOT EXISTS.

O gargalo remanescente, um `Nested Loop` que demora 1.28s, representa agora o custo computacional real e inevitável de processar os 80 570 "colegas de estrela" do utilizador `christopherharmon25`.

2.4.5. Resultados Finais

A tabela resume o impacto das alterações aplicadas, comparando os tempos médios da versão *baseline* com a versão final (índice + NOT EXISTS).

argumento (*username*)	tempo base (s)	tempo final (s)
christopherharmon25	19.119 s	1.920 s
rdavila66	10.425 s	0.217 s
jenkinsbrandon52	9.097 s	0.012 s

Em conclusão, a reformulação da *query* 4 revela-se claramente eficaz, demonstrando como uma abordagem em duas fases (primeiro resolvendo o gargalo de I/O com um índice estratégico e depois ajustando a lógica da consulta com NOT EXISTS) reduziu o tempo de execução em cerca de 90% (aproximadamente 10× mais rápido) para o caso de teste principal.

2.4.6. Comparação com DuckDB

Os testes no `DuckDB` foram executados diretamente sobre os dados originais, medindo o tempo de resposta wall-clock para garantir consistência nas métricas. A Tabela 5 apresenta os resultados obtidos para a query 4.

argumento	**PostgreSQL** (optimizado)	**DuckDB** (sem optimização)	**diferença**
christopherharmon25	1.920 s	1.395 s	DuckDB −27%
rdavila66	0.217 s	1.344 s	postgreSQL −84%
jenkinsbrandon52	0.012 s	1.137 s	postgreSQL −98%

Tabela 5: Comparação de desempenho: PostgreSQL optimizado vs. DuckDB sem optimização.

A análise dos resultados evidencia duas realidades distintas:

1. **Cargas pesadas (agregação):** no cenário mais exigente (`christopherharmon25`), o DuckDB superou o PostgreSQL (1.4s vs 1.9s). A análise do plano de execução revelou que o DuckDB recorreu a força bruta, varrendo sequencialmente 160 milhões de registos na tabela `star`. A sua vantagem reside na arquitetura colunar, que lê apenas os dados estritamente necessários do disco.
2. **Acessos pontuais (latência):** nos cenários de alta seletividade (rdavila66 e jenkinsbrandon52), o PostgreSQL apresenta tempos consideravelmente inferiores (0.01 s vs 1.14 s). O índice B-Tree criado permite validar rapidamente a existência de registos, com complexidade $O(\log N)$. O DuckDB, sem suporte a índices, precisa de varrer a tabela para confirmar presença ou ausência de dados, incorrendo sempre num custo fixo inicial.

Em síntese, a arquitetura colunar do DuckDB proporciona um desempenho base muito competitivo em cargas analíticas de grande volume, enquanto o PostgreSQL (através de índices e do `NOT EXISTS`) garante vantagens claras em consultas que exigem latências reduzidas e acessos altamente seletivos.

3. Carga Transacional

A componente transacional do trabalho prático simula um cenário de utilização intensiva de uma plataforma de gestão de repositórios. O objetivo desta fase consistiu na maximização do débito (*throughput*) e minimização da latência das operações (*response time*).

No cenário inicial, sem quaisquer estruturas de optimização, o sistema registou um desempenho de aproximadamente **78 transações por segundo (txn/s)**, com um **tempo de resposta superior a 200 ms**. A análise dos tempos de execução revelou que a latência era dominada por três operações de leitura, que consumiam excessivos recursos de CPU e I/O devido a varrimentos completos das tabelas e ordenações em memória.

3.1. Optimização através de Índices

A estratégia de indexação focou-se nas colunas utilizadas em cláusulas de filtragem (`WHERE`), junção (`JOIN`) e ordenação (`ORDER BY`) das interrogações críticas. Após análise dos planos de execução gerados pelo PostgreSQL através do comando `EXPLAIN ANALYZE`, identificaram-se os seguintes gargalos:

- **pesquisa textual (`SearchRepositories`):** a operação recorria a um *scan* sequencial para calcular vetores de texto em tempo de execução. A implementação de um índice invertido (GIN) na coluna `description` da tabela `repository` permitiu substituir o varrimento por uma pesquisa indexada eficiente:

```
CREATE INDEX idx_repository_description_gin
ON repository USING GIN(to_tsvector('english',description));
```

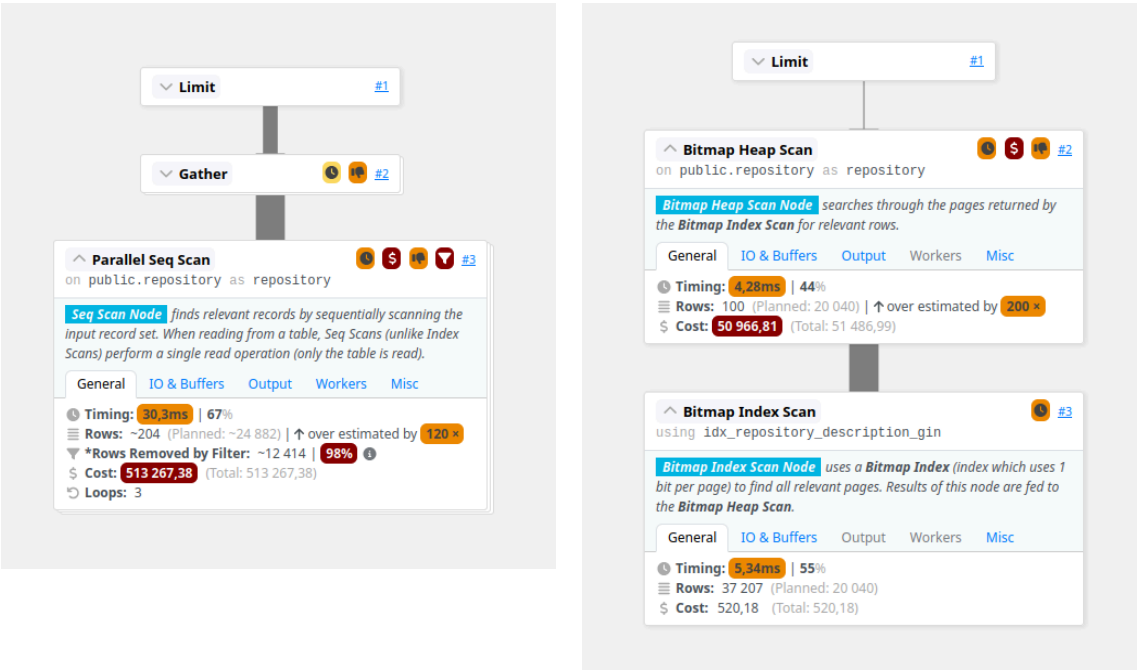


Figura 10: Comparação do plano de execução da query `SearchRepositories` antes e depois da criação do índice GIN na coluna `description`.

- **perfil de utilizador (`GetUserInfo`):** esta interrogação filtrava repositórios por proprietário e ordenava-os por data. A criação de um índice composto permitiu satisfazer ambos os requisitos simultaneamente, eliminando a necessidade de uma etapa de ordenação explícita (*Sort*):

```
CREATE INDEX idx_repository_owner_pushed_at
ON repository(owner,pushed_at DESC);
```

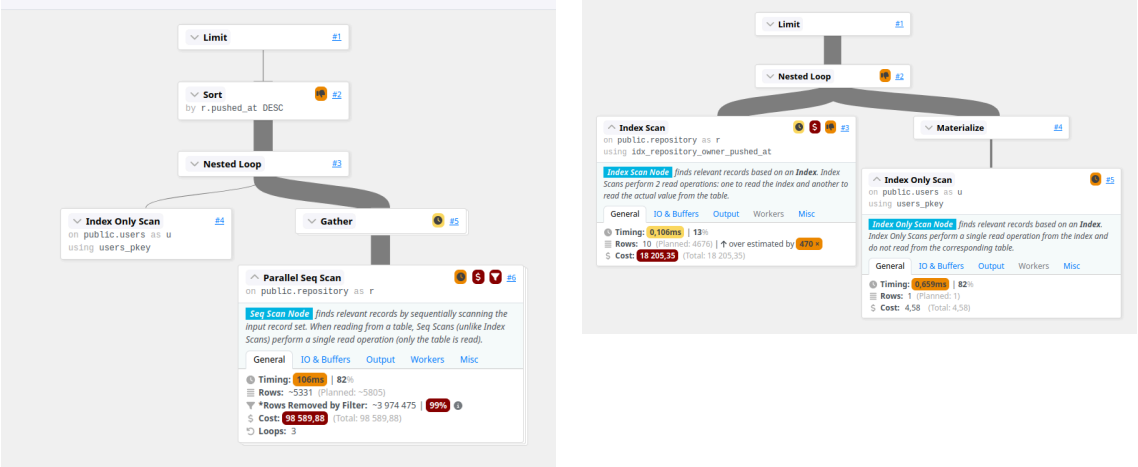


Figura 11: Impacto do índice composto `idx_repository_owner_pushed_at` na query `GetUserInfo`, eliminando a operação de ordenação (Sort).

- **listagem por linguagem (`GetRepositoriesByLanguage`):** identificou-se que a junção com a tabela `repository_languages` era o principal gargalo. A criação de um índice na chave estrangeira `language_id` otimizou significativamente a operação de junção:

```
CREATE INDEX idx_repository_languages_language_id
ON repository_languages(language_id);
```

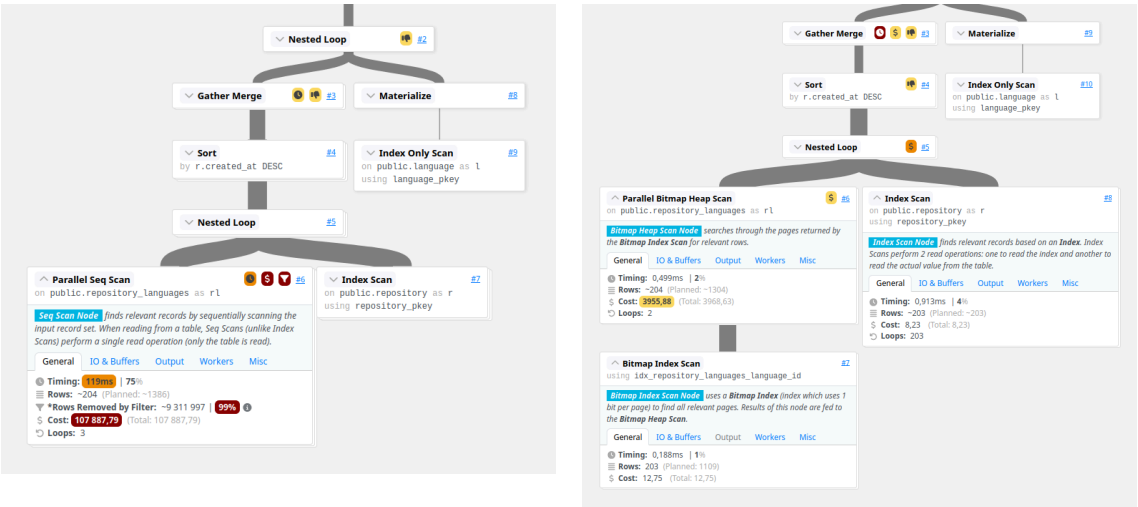


Figura 12: Optimização da junção na query `GetRepositoriesByLanguage` através da indexação da chave estrangeira `language_id`.

Durante a fase de testes, explorou-se a criação de um índice adicional na coluna `created_at` da tabela `repository` com o intuito de otimizar a ordenação temporal. Contudo, observou-se uma degradação de desempenho. A análise do plano de execução revelou que o optimizador do PostgreSQL, ao utilizar este índice, optava por um *Index Scan* seguido de *Nested Loops*. Para linguagens com baixa cardinalidade (poucos repositórios), esta estratégia obrigava o motor de base de dados a ler e descartar milhares de registos aleatórios até encontrar o número suficiente de correspondências, revelando-se menos eficiente do que a ordenação em memória (*Sort*) dos resultados filtrados. Consequentemente, optou-se por não incluir este índice na solução final.

A aplicação dos índices selecionados resultou num aumento de débito superior a 10 vezes em relação à configuração base, conforme demonstrado na Tabela 6.

métrica	sem índices	com índices	melhoria
throughput (txn/s)	77.81	812.07	+943.6%
SearchRepositories (ms)	623.71	31.57	−94.9%
GetRepositoriesByLanguage (ms)	1914.81	291.22	−84.8%
GetUserInfo (ms)	1447.47	1.62	−99.9%

Tabela 6: Comparação de *performance*: baseline vs. versão com índices.

3.2. Optimização de Parâmetros de Configuração

Após a estabilização do esquema físico através da criação de índices, procedeu-se à otimização dos parâmetros de configuração do PostgreSQL. A estratégia de otimização seguiu uma abordagem hierárquica, começando pela alocação de memória (o recurso mais crítico), passando pelo paralelismo de CPU, e terminando no ajuste fino das garantias de durabilidade (I/O).

3.2.1. shared_buffers

O parâmetro `shared_buffers` é responsável por estipular o volume de memória que o servidor reserva para o armazenamento em *cache* de blocos de dados. Um valor adequado reduz a necessidade de leituras frequentes ao disco, mantendo os dados mais acedidos em memória. Verificando a configuração inicial do sistema, obteve-se:

```
git=# SHOW shared_buffers;
 shared_buffers
----------------
 128MB
(1 row)
```

Executando o comando `free -m` na máquina virtual, foi possível verificar a disponibilidade de recursos:

total (MB)	used (MB)	free (MB)	shared (MB)	buff/cache (MB)
15980	708	1864	2113	13407

Tabela 7: Informação de memória disponível no sistema.

Com aproximadamente 15GB de memória RAM disponível, o valor por defeito de 128MB é manifestamente insuficiente. A documentação do PostgreSQL recomenda valores entre 20% e 25% da memória total do sistema para este parâmetro. Realizou-se uma análise de sensibilidade testando valores progressivos (1GB, 2GB, 4GB e 8GB) para determinar o ponto de equilíbrio ideal.

A Tabela 8 apresenta os resultados obtidos para cada configuração testada, enquanto a Figura 13 ilustra graficamente a evolução do desempenho.

configuração	throughput (txn/s)	latência média (ms)	variação
128 MB (base)	812.07	19.57	—
1 GB	956.63	16.61	+17.8%
2 GB	980.40	16.37	+2.5%
4 GB	1085.88	14.72	+10.7%
8 GB	1001.40	15.97	−7.8%

Tabela 8: Impacto da alocação de `shared_buffers` no desempenho transacional.

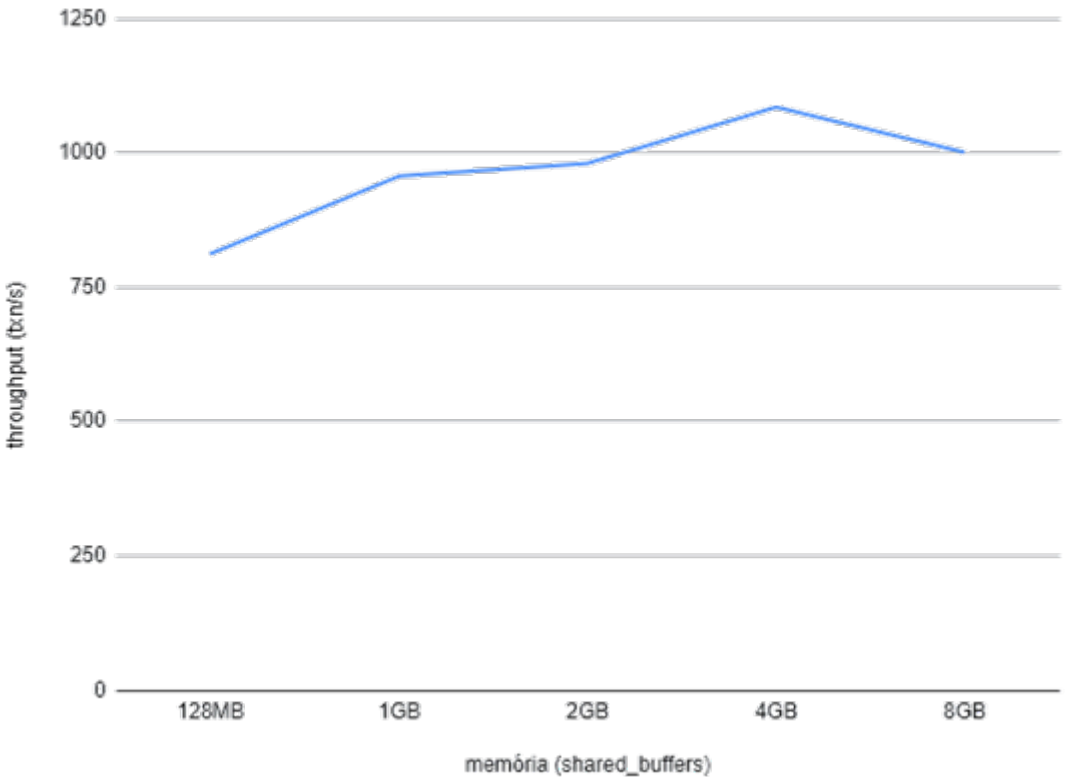


Figura 13: Evolução do *throughput* em função do `shared_buffers`.

Observou-se um crescimento consistente do desempenho até à configuração de 4GB, onde se atingiu o pico de 1085 txn/s e a latência mínima de 14.72 ms. Este valor corresponde a aproximadamente 25% da memória total do sistema, alinhando-se com as recomendações gerais da documentação do PostgreSQL.

Contudo, ao aumentar a alocação para 8GB (50% da RAM), registou-se uma regressão de desempenho, com o débito a cair para 1001 txn/s e a latência a aumentar para 15.97 ms. Este fenómeno justifica-se pelo mecanismo de *dupla cache* ("double buffering"). O PostgreSQL depende não só do seu `shared_buffers`, mas também da *cache* de sistema de ficheiros do sistema operativo (OS Cache) para gerir escritas e leituras que excedem o *buffer* interno. Ao alocar uma porção excessiva de RAM exclusivamente ao PostgreSQL (8GB), reduziu-se a memória disponível para o sistema operativo gerir ligações, processos concorrentes e a sua própria *cache* de ficheiros, resultando numa contenção de recursos que prejudicou o desempenho global.

Conclui-se, portanto, que **4GB** é o valor ideal para este cenário, maximizando a utilização de *cache* interna sem comprometer o equilíbrio de recursos do sistema.

3.2.2. work_mem

O parâmetro `work_mem` define a quantidade máxima de memória que **cada** operação interna de ordenação (*sort*) ou *hash* pode utilizar antes de recorrer a ficheiros temporários em disco. Este parâmetro é particularmente relevante em *joins* (ex.: `Hash Join`), agregações, operações de ordenação e também em alguns cenários com `Bitmap Heap Scan`, onde estruturas intermédias podem tornar-se mais pesadas e levar a trabalho adicional.

Verificando a configuração inicial:

```
git=# SHOW work_mem;
4MB
```

A Tabela 9 apresenta os resultados obtidos com o `EXPLAIN ANALYZE` para a configuração-base de 4MB.

query	operação principal	planning (ms)	execution (ms)
CreateRepository	*Insert*	0.097	1.126
AddStar	*Insert*	0.013	0.095
GetRepositoryPage	*Index Scan*	1.273	0.389
GetReposbyLanguage	*Nested Loop*	0.196	5.829
GetUserProfile	*Index Scan*	0.128	0.023
GetUserRepos	*Nested Loop*	0.081	0.065
GetTicketPage	*Index Scan*	0.151	0.013
SearchRepository	*Bitmap Heap Scan*	0.070	11.373

Tabela 9: Resultados do EXPLAIN ANALYZE com work_mem = 4MB.

Analisando os resultados da Tabela 9, retiram-se as seguintes conclusões:

- **pesquisa textual**: a query `SearchRepository` destaca-se agora como a mais lenta (~11.4ms), utilizando um `Bitmap Heap Scan`. Com apenas 4MB, a gestão dos **bitmaps** em memória para os resultados da pesquisa textual (`to_tsvector`) pode ser um fator limitante, tornando-a a principal candidata a optimização.
- ***joins* e agregações**: a query `GetReposbyLanguage` (~ 5.8ms) utiliza `Nested Loop`. Este comportamento sugere que o otimizador não encontrou benefício suficiente (ou condições) para optar por estratégias baseadas em *hash* (ex.: `Hash Join`), ou que os custos associados às alternativas não compensam no contexto atual.
- **inserções**: `CreateRepository` (~ 1.1ms) mostra-se eficiente. Como esperado, `work_mem` tende a ter impacto reduzido em escritas simples, cujo custo é normalmente dominado por WAL, *locks* e I/O.
- **leituras rápidas**: queries como `GetTicketPage` e `GetUserProfile` apresentam tempos vestigiais (< 0.1ms), confirmando que os índices e o acesso pontual (`Index Scan`) estão a funcionar corretamente.

Com base nesta análise, aumentou-se o `work_mem` para **16MB** e **32MB**, focando a atenção na evolução das queries `SearchRepository` e `GetReposbyLanguage`.



Figura 14: Impacto do aumento de work_mem (4MB, 16MB e 32MB) no tempo de execução das queries.

Analisando os resultados globais apresentados na Figura 14, observam-se os seguintes comportamentos:

- **SearchRepository**: esta query apresenta a melhoria mais significativa, com uma redução de ~22% no tempo de execução ao aumentar de 4MB (~11.3ms) para 32MB (~8.8ms). O aumento para 16MB já proporciona ganhos notáveis (~10.7ms), indicando que operações com `Bitmap Heap Scan` e filtragem textual beneficiam de maior disponibilidade de memória para processamento intermédio.
- **GetReposbyLanguage**: observa-se uma melhoria modesta mas consistente, com tempos a reduzirem de ~5.8ms (4MB) para ~5.3ms (32MB). Embora o optimizador continue a utilizar `Nested Loop` em vez de `Hash Join`, a memória adicional permite uma gestão mais eficiente das operações intermediárias e do *buffer* de dados durante a execução dos *joins*.
- **operações sem impacto significativo**: `Add Star`, `Get Repository Page`, `Get Ticket Page`, `Get User Profile` e `Get User Repos` mantêm tempos praticamente constantes independentemente do `work_mem`. Isto confirma que estas operações, dominadas por `Index Scan` ou inserções simples, não são limitadas por este parâmetro.

De forma agregada, os testes demonstram que o aumento de `work_mem` melhora sobretudo operações específicas:

1. **pesquisas textuais** beneficiam significativamente, com reduções até 22% no tempo de execução.
2. ***joins* e agregações** apresentam melhorias moderadas (5%-10%), sugerindo que outros fatores (cardinalidade, seletividade, desenho de índices e distribuição de dados) podem ser determinantes para ganhos adicionais.
3. **ponto de retornos decrescentes**: a diferença entre 16MB e 32MB é menor do que entre 4MB e 16MB, indicando que aumentos adicionais tendem a produzir ganhos marginais.

Assim, **recomenda-se a configuração `work_mem = 16MB`** como um compromisso equilibrado entre *performance* e a utilização de memória. Este valor permite capturar a maior parte dos ganhos observados (em particular na interrogação `SearchRepository`) sem introduzir o risco acrescido de pressão sobre a memória associado a 32MB, sendo, por isso, mais adequado a ambientes com múltiplas interrogações concorrentes.

3.2.3. effective_cache_size

O parâmetro `effective_cache_size` fornece ao *planner* do PostgreSQL uma estimativa da quantidade de dados que pode estar em cache durante a execução de queries. Este valor **não** reserva memória real, influenciando apenas o modelo de custos do optimizador. Em termos práticos, `effective_cache_size` representa uma estimativa da cache efetivamente disponível para leituras, tipicamente englobando o `shared_buffers` do PostgreSQL e a *page cache* do sistema operativo. Valores mais altos tendem a reduzir o custo estimado de acessos aleatórios, favorecendo a utilização de índices em detrimento de varrimentos sequenciais (*sequential scans*) quando existe ambiguidade na escolha do plano.

Dado que a máquina utilizada dispõe de 16GB de RAM, e tendo identificado a transação `SearchRepositories` (pesquisa de texto) como uma das mais custosas no *benchmark* transacional, focámos a análise no impacto deste parâmetro no tempo de execução desta *query* específica.

Foram testados três cenários de configuração para o `effective_cache_size`: **4GB**, **8GB** e **12GB**. Para cada configuração, executou-se a *query* de pesquisa textual utilizando `EXPLAIN (ANALYZE,BUFFERS)` para medir o tempo de execução e analisar o plano escolhido pelo optimizador.

A *query* analisada foi:

```
SELECT id, name, owner, description
FROM repository
WHERE to_tsvector('english',description) @@ to_tsquery('english','java')
LIMIT 100;
```

A Tabela 10 apresenta os tempos médios obtidos nas três configurações (a variação observada é pequena e compatível com ruído normal do sistema, já que o plano se manteve inalterado):

configuração	tempo de execução (ms)
4 GB	13.964
8 GB	13.506
12 GB	13.481

Tabela 10: Tempos de execução da query `SearchRepositories` para diferentes tamanhos de cache.

A análise do `EXPLAIN` revelou que, em todos os cenários, o PostgreSQL escolheu o mesmo tipo de estratégia: um `Bitmap Index Scan` sobre o índice GIN (`idx_repository_description_gin`), seguido de `Bitmap Heap Scan` para obter os tuplos no *heap*. Ou seja, variar `effective_cache_size` não foi suficiente para alterar a decisão do optimizador (índice vs *seq scan*), mantendo-se o mesmo plano lógico.

Além disso, a análise dos buffers revelou que a maior parte dos blocos necessários foi servida diretamente a partir da cache interna do PostgreSQL (ou seja, os `shared_buffers`), conforme evidenciado pelos contadores de *shared hit* no output do EXPLAIN.

```
Buffers: shared hit=408
...
Heap Blocks: exact=75
Buffers: shared hit=333
```

Ao contrário do que seria expectável em cenários onde o índice não cabe em memória, a alteração do `effective_cache_size` **não teve impacto significativo** no desempenho desta interrogação, com os tempos a estabilizarem em torno dos 13.5 ms (a ligeira variação observada insere-se na margem de ruído normal do sistema).

Este resultado explica-se por dois fatores principais:

1. **plano já "dominante" com GIN**: para esta pesquisa (`to_tsvector @@ to_tsquery`), o índice GIN apresenta uma seletividade suficientemente forte para que o *planner* prefira sempre a estratégia baseada em índice, mesmo com uma estimativa conservadora de cache (4GB). Aumentar para 12GB apenas reforça uma decisão que já era claramente a mais barata no modelo de estimativa de custos.
2. **dados em `Shared Buffers`:** a predominância de shared hits indica que tanto o índice como os blocos relevantes do heap já se encontravam nos `shared_buffers`. Como não houve alteração no plano de execução e a maioria dos acessos não implicou leituras físicas, o parâmetro `effective_cache_size` teve pouca influência na latência observada para esta operação.

Conclui-se que, para o volume de dados atual e para este perfil de pesquisa textual, variar o valor de `effective_cache_size` não altera o plano nem produz ganhos relevantes. Assim, a configuração base de memória já é suficiente para garantir a escolha do plano de execução ideal.

3.2.4. Paralelismo

O PostgreSQL suporta execução paralela de queries, permitindo acelerar operações de leitura intensiva (por exemplo, `Parallel Seq Scan`, alguns tipos de agregação e junções com volumes elevados). Este mecanismo cria *background workers* adicionais, coordenados por um processo-líder, o que pode reduzir o tempo de execução quando o custo de processamento compensa o overhead de coordenação e sincronização. Numa máquina com 8 vCPUs, o objetivo é encontrar um ponto de equilíbrio entre paralelismo intra-query e o número total de workers ativos no servidor, evitando saturação de CPU e degradação por contenção.

Para ajustar o paralelismo, focámo-nos em dois parâmetros:

max_parallel_workers_per_gather: número máximo de *workers* que podem ser utilizados por cada nó `Gather`/`Gather Merge` de um plano. Na prática, este parâmetro define o teto do paralelismo **por query**, embora uma query possa ter mais do que um nó `Gather`.

max_parallel_workers: número máximo global de *parallel workers* que podem estar ativos simultaneamente no servidor. Este valor limita o paralelismo total do sistema sob concorrência, independentemente do valor configurado por query.

Antes de aplicar alterações globais, avaliou-se o impacto do paralelismo numa interrogação de leitura intensiva, `GetRepositoriesByLanguage`, que envolve junções e ordenação. Variou-se o número máximo de workers por query e mediu-se o tempo de execução via `EXPLAIN ANALYZE`:

workers (per_gather)	tempo de execução (ms)	melhoria (%)
2 (default)	48.76	-
3	38.06	~ 22%
4	37.62	~ 23%
5	38.15	~ 22%

Tabela 11: Impacto do `max_parallel_workers_per_gather` no tempo de execução da query GetRepositoriesByLanguage.

Observa-se um ganho substancial ao aumentar de 2 para 3 workers, o que indica que existia trabalho paralelizável suficiente para compensar o custo de coordenação. A partir de 3 workers, verifica-se a lei dos rendimentos decrescentes: a diferença entre 3 e 4 é marginal e pode situar-se perto do ruído experimental, enquanto 5 workers já não traz benefício e até piora ligeiramente. Assim, selecionou-se `max_parallel_workers_per_gather = 4` como valor de referência, por representar o melhor valor observado sem inflacionar excessivamente o paralelismo por query.

De seguida, realizaram-se testes sob carga transacional com várias combinações de `max_parallel_workers_per_gather` e `max_parallel_workers`, medindo-se throughput e tempo médio de resposta. O objetivo foi avaliar não apenas o desempenho de uma query isolada, mas o comportamento do sistema quando múltiplas sessões competem por CPU e por workers paralelos.

configuração (per_gather,global)	throughput (txn/s)	response time (ms)
4,8	1189.06	13.47
4,16	1163.70	13.80
4,32	1128.23	14.20
2,32	1100.21	14.51

Tabela 12: Impacto de `max_parallel_workers_per_gather` e `max_parallel_workers` no desempenho transacional.

A análise da Tabela 12 evidencia dois padrões relevantes. Primeiro, mantendo o limite global de workers elevado (32), o aumento do paralelismo por query de 2 para 4 workers traduz-se num aumento do throughput e numa redução da latência, indicando que as interrogações mais exigentes do benchmark beneficiam de um maior grau de paralelismo quando existem recursos disponíveis. Segundo, o melhor desempenho absoluto é observado na configuração (4,8), que supera configurações com limites globais superiores (16 e 32), apesar de estas permitirem, em teoria, um maior número de workers.

Este comportamento pode ser explicado por dois efeitos típicos em cenários de carga elevada em máquinas com CPU limitada. Por um lado, limites globais excessivamente elevados intensificam a competição por CPU, aumentando o overhead de escalonamento e reduzindo a previsibilidade do tempo de resposta. Por outro, quando múltiplas sessões tentam explorar paralelismo em simultâneo, o sistema pode acabar por atribuir menos workers por interrogação do que o configurado, devido à indisponibilidade de workers livres, desperdiçando assim parte do custo de planeamento e coordenação sem alcançar o ganho de desempenho paralelo esperado.

Com base nestes resultados, determinou-se como configuração recomendada para este ambiente `max_parallel_workers_per_gather = 4` e `max_parallel_workers = 8`. Esta combinação maximiza o benefício em queries complexas sem exceder a capacidade de processamento das 8 vCPUs, produzindo simultaneamente o maior throughput e a menor latência média observados nos testes.

3.2.5. fsync

O parâmetro *fsync* controla se o PostgreSQL força a escrita efetiva de WAL e ficheiros de dados no armazenamento persistente, garantindo que uma transação só é considerada concluída quando a informação necessária para a sua durabilidade foi efetivamente persistida. Com `fsync = off`, o servidor deixa de emitir operações de sincronização para disco, delegando essa responsabilidade no *page cache* do sistema operativo, o que elimina garantias de durabilidade e pode conduzir a corrupção de dados após falhas de energia, *crashes* do kernel ou reinícios inesperados.

Nesta secção avaliou-se o impacto desta alteração, comparando o desempenho com a configuração-base optimizada anteriormente (`max_parallel_workers_per_gather = 4` e `max_parallel_workers = 8`), mantendo o restante ambiente e carga constantes.

configuração	throughput (txn/s)	response time (ms)	variação (throughput)
fsync ON (base)	1189.06	13.47	-
fsync OFF	1138.41	13.63	−4.3%

Tabela 13: Impacto da desativação do *fsync* no desempenho transacional.

Para compreender este resultado, analisaram-se também os tempos médios por query:

query	tempo base (ms)	tempo fsync="off" (ms)	variação
CreateRepository	1.802	0.709	−60.7%
StarRepository	2.092	1.112	−46.8%
CreateCommit	2.858	1.687	−41.0%
CreateTicket	1.921	0.805	−58.1%
CloseTicket	2.209	1.067	−51.7%
GetRepositoryInfo	7.753	7.078	−8.7%
GetRepositoriesByLanguage	174.007	191.181	+9.9%
GetUserInfo	1.357	1.467	+8.1%
GetTicketInfo	0.875	0.921	+5.3%
SearchRepositories	34.969	37.623	+7.6%

Tabela 14: Tempo de resposta por query com fsync=off.

Os resultados mostram um comportamento assimétrico. As operações de escrita obtêm ganhos expressivos (por exemplo, `CreateRepository` e `CreateTicket`), o que é consistente com a remoção da latência associada a sincronização para armazenamento persistente. No entanto, várias operações de leitura (e, em particular, leituras pesadas como `GetRepositoriesByLanguage` e `SearchRepositories`) degradam, o que explica a redução do throughput global observada na Tabela 13.

Uma explicação plausível para esta degradação é o efeito de interferência entre escrita e leitura no subsistema de I/O e na gestão de memória do sistema operativo. Com `fsync = off`, o volume de dados "sujos" (*dirty pages*) tende a acumular no *page cache* e é posteriormente descarregado para disco por mecanismos de *writeback* do kernel em rajadas, podendo criar períodos de saturação de I/O e maior latência para leituras concorrentes. Em paralelo, a pressão de memória causada por mais páginas sujas pode aumentar a expulsão de dados úteis da cache, elevando a probabilidade de leituras irem ao disco e agravando ainda mais as queries de leitura pesada.

Conclui-se que `fsync = off` não só remove uma garantia crítica de durabilidade como também se revelou contraproducente em desempenho agregado (redução de 4.3% no throughput). Assim, **recomenda-se manter `fsync = on`**.

3.2.6. synchronous_commit

O parâmetro synchronous_commit define o grau de sincronização exigido no processamento do WAL (Write-Ahead Log) antes de o servidor confirmar o sucesso de uma transação ao cliente. No modo normal (valor por omissão, on), a confirmação do commit só é devolvida após o flush do WAL para armazenamento persistente. Quando configurado como synchronous_commit = off, esta espera é eliminada, permitindo que o servidor reporte o sucesso da transação assim que esta fica logicamente concluída.

Esta configuração introduz um risco limitado de perda das transações mais recentes em caso de crash, correspondente à janela temporal entre a confirmação ao cliente e a persistência efetiva do WAL. No entanto, não compromete a consistência nem a integridade da base de dados: no pior cenário, o estado recuperado

após falha é semanticamente equivalente ao de essas transações nunca terem sido confirmadas, isto é, terem sido abortadas.

A alteração foi testada sobre a configuração-base optimizada (4 workers por query e 8 workers globais), mantendo a carga e o ambiente constantes.

configuração	throughput (txn/s)	response time (ms)	variação
sync_commit ON (base)	1189.06	13.47	-
sync_commit OFF	1075.88	14.77	−9.5%

Tabela 15: Impacto do synchronous_commit no desempenho transacional.

query	tempo base (ms)	tempo sync_commit=off (ms)	variação
CreateRepository	1.802	0.784	−56.5%
StarRepository	2.092	1.095	−47.7%
CreateCommit	2.858	1.862	−34.8%
CreateTicket	1.921	0.884	−54.0%
CloseTicket	2.209	1.189	−46.2%
GetRepositoryInfo	7.753	7.400	−4.6%
GetRepositoriesByLanguage	174.007	198.557	+14.1%
GetUserInfo	1.357	1.567	+15.5%
GetTicketInfo	0.875	1.073	+22.6%
SearchRepositories	34.969	49.559	+41.7%

Tabela 16: Tempo de resposta por query com synchronous_commit=off.

A desativação do *synchronous_commit* produziu um *trade-off* claro entre latência de escrita e desempenho global. As operações de escrita beneficiaram significativamente (por exemplo, `CreateTicket` reduziu de 1.92 ms para 0.88 ms e `CreateCommit` de 2.86 ms para 1.86 ms), o que é consistente com commits assíncronos que deixam de bloquear à espera do *flush* do WAL.

Apesar disso, o throughput global caiu 9.5% (Tabela 15), acompanhado por um aumento relevante no tempo de resposta de queries de leitura pesada (Tabela 16), com destaque para `GetRepositoriesByLanguage` (+14%) e `SearchRepositories` (+42%). Uma interpretação plausível é que, ao deslocar parte do trabalho de persistência para flushing assíncrono, o sistema fica mais sujeito a interferência entre escrita e leitura (contenção de CPU/I/O e maior variabilidade), o que penaliza desproporcionalmente as queries mais longas e acaba por reduzir o débito agregado.

Embora `synchronous_commit = off` reduza a latência de escritas isoladas, neste *workload* degradou o desempenho global e introduz um risco de perda das transações mais recentes em caso de *crash*. Por isso, **recomenda-se manter `synchronous_commit = on`** como configuração default.

4. Conclusão

O presente trabalho evidenciou a relevância de uma abordagem estruturada à otimização de bases de dados relacionais, cobrindo cargas analíticas e transacionais. Com base na análise de planos de execução, foram identificados gargalos e aplicadas estratégias de indexação, reestruturação de queries e ajustes de configuração. Os resultados apontam para ganhos consistentes de desempenho, reforçando o impacto de decisões informadas no desenho e na configuração do sistema.